Zensar Content Studio

Installation and User Manual

This manual explains how to start Zensar Content Studio on a computer where it is already installed (for example the cloud GPU server), and how to set it up from scratch on a new Windows computer. The application has a FastAPI backend (the engine, written in Python) and a React frontend (the screens you see in the browser). Both are started for you by one file, `start.bat`. The local AI runs through Ollama, and pictures are created by a local image model. Nothing is sent to the internet.

> Run every command from the **project folder**: the folder that contains `start.bat`, `backend\` and `frontend\`.
>
> **Project files location:** the code is on GitHub at **https://github.com/V-JasonEmmanuel/Zen-TMT**. On a computer where it is installed, the project folder is wherever it was cloned or extracted, for example `D:\Zen-TMT`. In the examples below, replace `D:\Zen-TMT` with your own folder.

If Zensar Content Studio is already installed on the computer, together with its prerequisites (Python, the Python packages, the AI models and Ollama), you do **not** need the fresh installation steps (sections 1–10). Follow sections A1–A8 each time you want to start the application.

Contents

- A1. Before you start
- A2. Open the project folder
- A3. Start the supporting services
- A4. Start the application
- A5. Use the application
- A6. Stop the application
- A7. Open the application from another computer
- A8. Common problems
- 1. What needs to be installed
- 2. Download the project
- 3. Run the one-click installer
- 4. Configure the application
- 5. Local AI models
- 6. Install behind a corporate proxy
- 7. Start the backend and frontend separately (developer mode)

8. Use the application
9. Run tests
10. Stop and restart
Troubleshooting
11. Project File Reference
12. Sample Data Information

A1. Before you start

- Log in to the computer or cloud server (for a cloud server, use **Remote Desktop**) with your username and password.
- Have the project folder path ready, for example `D:\Zen-TMT`. It is the folder that contains `start.bat`.
- Open the web browser **on that computer itself** (for example inside your Remote Desktop session). The application listens on `127.0.0.1` / `localhost`, so another computer's browser cannot open it unless you follow section A7.
- You need only **one** window. `start.bat` starts the backend and the frontend together; you do not need separate terminals.

A2. Open the project folder

1. Press **Windows key + E** to open **File Explorer**.
2. Go to the project folder, for example `D:\Zen-TMT`.

Or open **Windows PowerShell** and move to the folder:

```
cd D:\Zen-TMT
dir
```

The listing should show `start.bat`, `stop.bat`, `backend` and `frontend`. If it does not, you are in the wrong folder. If you see only one folder called `Zen-TMT-main`, open it: the files are inside.

> Do **not** delete the `data\` folder. It holds your projects, converted papers, learned brand templates and branded documents. The `.env` file holds your settings; keep it too.

A3. Start the supporting services

Ollama (the local writing AI). Ollama normally starts by itself with Windows and runs in the background (a llama icon near the clock). To confirm, run this in PowerShell and check that a model such as `qwen2.5:7b` or `qwen2.5:3b` is listed:

```
ollama list
```

If Ollama is not running, start it in a separate PowerShell window and leave that window open:

```
ollama serve
```

If you do not start Ollama, the application still works. It then builds slides and shortens documents by selecting the document's own key sentences instead of rewriting them with the AI.

Image model. Nothing to start. The picture-making AI starts automatically when a branded document needs pictures, and closes again afterwards.

A4. Start the application

1. In the project folder, **double-click** `start.bat`.
2. A window titled **"Zensar Content Studio - setup"** opens. Because everything is already installed, each step shows **OK** and takes only seconds.
3. When you see **"Content Studio is running at http://localhost:8000"**, the browser opens the application automatically. If it does not, open the browser and type `http://localhost:8000`.
4. A small minimised window called **"Zensar Content Studio (close to stop)"** keeps the application running. **Leave it open** while you work.

You can also start it from the PowerShell window opened in A2:

```
.\start.bat
```

If Content Studio is already running, `start.bat` simply opens the browser again. If another program is using port 8000, it picks the next free port (8001, 8002, …) and shows the new address.

To check that the backend is running, open `http://127.0.0.1:8000/api/health` in the browser; it answers `{"ok":true}`. The interactive API documentation is at `http://127.0.0.1:8000/api/docs`.

A5. Use the application

1. Open the **Dashboard** and check the **System** panel: green dots mean the local AI, models and tools are ready.
2. To turn documents into slides, pictures and a video, click **New project**, add your files, choose a design, describe what you want and click **Generate**.
3. To reformat a research paper for a publisher, open **Research Papers**, upload the paper, choose a format (for example *IEEE* or *Springer Nature journal*) and click **Convert paper**.
4. To make a branded Zensar white paper, open **Brand Documents**, choose the *Zensar White Paper* template, upload your document and click **Create branded PDF**.
5. Every result can be downloaded from its page (PowerPoint, PDF, Word, LaTeX, MP4).

Section 8 explains every screen step by step.

A6. Stop the application

- Close the window called **"Zensar Content Studio (close to stop)"**, **or**
- double-click `stop.bat` in the project folder.

Ollama can be left running.

A7. Open the application from another computer

The application has no login, so by default only the computer it runs on can open it. To use it from your own PC while it runs on a cloud server, choose one:

Recommended: SSH tunnel

On your own PC, run the command below in PowerShell, then open `http://localhost:8000` on your PC:

```
ssh -L 8000:127.0.0.1:8000 <user>@<server-address>
```

Open the port

On the server, start the application from Command Prompt with the commands below. Then allow port 8000 in the cloud firewall **only from your own IP address**:

```
set ZCS_HOST=0.0.0.0
start.bat
```

A8. Common problems

Symptom	What to do
Browser shows "This site can't be reached"	The application is not running. Double-click `start.bat` and wait for "Content Studio is running".
Port 8000 is already in use	Nothing: `start.bat` picks the next free port and shows the address. To free port 8000, close the old window or run `stop.bat`.
Dashboard shows "Local AI unavailable"	Run `ollama list` to confirm a model is installed, or start Ollama with `ollama serve`. Then choose the model in **Settings**.
Creating pictures is slow	Without an NVIDIA graphics card each picture takes 1–2 minutes. On the GPU server it takes seconds.
Branded PDF says "Arial in place of Graphik"	Graphik is a licensed font. Copy its files into `brands\zensar\assets\fonts\`.
Setup window shows a red message	Read the message. Details are in the `logs\` folder (`pip.log`, `npm.log`, `server.log`). Fix the cause and run `start.bat` again; it continues where it stopped.

Other problems are covered in the Troubleshooting section later in this manual.

Fresh Installation:

> **Fresh installation only:** Sections 1–10 are for setting up Zensar Content Studio on a new Windows computer or server. Where it is already installed, use sections A1–A8 instead.

1. What needs to be installed

- A Windows 10 / 11 PC, or Windows Server 2019 / 2022, with **16 GB** of memory and about **15 GB** of free disk space.
- An internet connection for the **first** start only. Afterwards the application works fully offline. If the company network allows only GitHub, see section 6.
- **Optional:** an NVIDIA graphics card. Picture creation is about 10× faster, and the larger AI model is used.
- **Optional:** Git, to download the project with `git clone`.

You do **not** need to install anything else by hand. `start.bat` installs whatever is missing: the Visual C++ runtime, **Python 3.12**, the Python packages, **Node.js** (only if the screens need rebuilding), the AI models and **Ollama**. No paid API key is needed.

2. Download the project

With Git: open PowerShell, go to the drive where the project should live, and clone the repository:

```
D:
git clone https://github.com/V-JasonEmmanuel/Zen-TMT.git
cd Zen-TMT
```

As a ZIP file: open https://github.com/V-JasonEmmanuel/Zen-TMT in the browser, click the green **Code** button, then **Download ZIP**. In your **Downloads** folder, right-click **Zen-TMT-main.zip**, choose **Extract All…**, pick a short location such as `D:\Zen-TMT` and click **Extract**.

3. Run the one-click installer

Double-click `start.bat` in the project folder. If Windows shows *"Windows protected your PC"*, click **More info → Run anyway** (this appears only because the file was downloaded).

The setup window works through numbered steps. The first time, this takes about 10–30 minutes:

Step shown	What it does (in plain words)
`[0] Microsoft Visual C++ runtime`	Installs a small Windows component that the AI parts need (only if it is missing).
`[1] Python`	Finds or installs Python 3.12 and creates a private copy for the application (the `.venv` folder).
`[2] Python packages`	Downloads the building blocks the application uses (about 300 MB).
`[3] Configuration`	Creates the settings file `.env` from `.env.example`.
`[4] Document understanding model`	Downloads a small AI model that understands what text means (about 90 MB).
`[5] Web interface`	Prepares the screens. The ready-built screens in `frontend\dist` are used when Node.js is not available.
`[6] Image generation`	Installs the picture-making AI (the GPU version when an NVIDIA card is found) and its model (about 2 GB). It **asks first**: type **Y** and press **Enter**.
`[7] Local AI`	Installs **Ollama** and a language model: `qwen2.5:7b` with a GPU of 8 GB or more, otherwise `qwen2.5:3b`. It **asks first**.
`[8] System check`	Checks that everything needed is present.
`[9] Starting Content Studio`	Starts the application and opens the browser.

To install without any questions (for example on a server), run this in Command Prompt in the project folder:

```
set ZCS_ASSUME_YES=1 && start.bat
```

If a step fails, read the red message, fix the cause (usually the internet connection) and run `start.bat` again. Finished steps are skipped.

4. Configure the application

> **First-time installation:** the `data\` folder must be empty (or absent) when you set up the application for a new user. If it contains projects or templates from a previous installation that you do not want, delete it before starting.

`start.bat` creates the settings file `.env` from the example the first time. To create it by hand, in PowerShell:

```
Copy-Item .env.example .env
```

The settings you are most likely to change:

Setting	Meaning
`OLLAMA_HOST`	Address of Ollama on this computer, normally `http://127.0.0.1:11434`. Outside addresses are refused.
`OLLAMA_MODEL`	The language model, for example `qwen2.5:7b`. Leave empty to choose it in **Settings**.
`BACKEND_PORT`	The port of the application (default `8000`).
`MAX_UPLOAD_MB`	Largest file you can upload (default `100`).
`TTS_ENGINE`	Voice for video narration: `auto`, `sapi` (Windows voices), `piper` or `none`.
`OCR_ENABLED`	Read text from scanned pages (`true` / `false`).
`DATA_DIR`, `OUTPUT_DIR`, `MODELS_DIR`	Where uploads, results and AI models are stored.

`start.bat` also understands these optional switches. Set them in Command Prompt before running it, for example `set ZCS_PORT=8100`:

Switch	Effect
`ZCS_ASSUME_YES=1`	Answer "yes" to every question (unattended installation).
`ZCS_PORT=8000`	The port to use.
`ZCS_HOST=0.0.0.0`	Let other computers open the application (see A7).
`ZCS_NO_BROWSER=1`	Do not open the browser.
`ZCS_SKIP_AI=1`	Do not install Ollama or a language model.
`ZCS_SKIP_IMAGES=1`	Do not install picture creation (saves about 5 GB).
`ZCS_OFFLINE=1`	Always install from the offline bundle (section 6).

Restart the application after changing `.env`.

Brand font

The Zensar typeface **Graphik** is licensed and is not in the repository. Copy its font files into `brands\zensar\assets\fonts\` to use it; otherwise the brand's fall-back font Arial is used.

5. Local AI models

`start.bat` installs these for you. To install them by hand:

Language model (Ollama). Install Ollama from https://ollama.com, then download a model:

```
ollama pull qwen2.5:7b
```

Use `qwen2.5:3b` on computers without a graphics card of 8 GB or more. Choose the model in **Settings**, or set `OLLAMA_MODEL` in `.env`. Optionally, `ollama pull qwen2.5vl:3b` lets the application understand demo videos frame by frame.

Text understanding and image models:

```
.venv\Scripts\python.exe scripts\fetch_models.py
.venv\Scripts\python.exe scripts\fetch_models.py --images
```

6. Install behind a corporate proxy

A proxy set in Windows or Git is detected automatically and passed on. If the network still blocks pypi.org (the Python package site), `start.bat` switches to the **offline bundle**, https://github.com/V-JasonEmmanuel/Zen-TMT-offline. It is downloaded with Git, the same way the project itself was cloned, and contains:

- every Python package for Windows and Python 3.12, including the GPU version of PyTorch;
- the text-understanding and image models;
- the Visual C++ runtime.

The pieces are put back together and checked (SHA-256) before anything is installed. Use **Python 3.12** on such a computer.

Ollama and its language model come from ollama.com, which is not in the bundle. Install them on a computer that can reach ollama.com and copy the folder `%USERPROFILE%\.ollama\models` across, or ask IT to allow ollama.com and registry.ollama.ai.

7. Start the backend and frontend separately (developer mode)

Normally `start.bat` does this for you. Developers who change the code can run the two parts in two terminals, with automatic reload.

First terminal (backend):

```
.venv\Scripts\Activate.ps1
python backend\main.py
```

The API runs on `http://127.0.0.1:8000`. If PowerShell will not activate `.venv`, use Command Prompt and run `.venv\Scripts\activate.bat`.

Second terminal (frontend):

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:3000`. Its `/api` requests are passed to the backend on port 8000. Keep both terminals running. To rebuild the ready-made screens in `frontend\dist`, run `npm run build`.

8. Use the application

The left-hand menu has these sections.

Dashboard

The home screen. It shows your recent projects, recent documents and the **System** panel: whether the local AI, the models and the tools are ready. Click **Settings** if something shows as missing.

New project: documents into slides, visuals and a video

1. Click **New project**.
2. **Sources:** drag your files into the box: documents (PDF, Word, PowerPoint, text), demo videos (.mp4) or a zipped code repository. You can also paste a GitHub address.
3. **Brand & template:** choose a design, for example Corporate, Bold or Minimal.
4. **Instruction:** describe what you want, for example *"10-slide executive summary for the leadership team"*.
5. **Outputs & video:** choose the narration voice, music and any extras.
6. Click **Generate**. A progress bar shows each stage.
7. When it finishes, the project opens with these tabs:
    - **Presentation:** view, present full-screen, download PPTX/PDF, edit or regenerate any slide.
    - **Images:** the generated diagrams and charts.
    - **Video editor:** arrange clips, trim, change narration, render the MP4.
    - **Sources:** which page each statement came from.
    - **Content plan / Files:** the plan and every output file.

Projects

A list of all projects. Click one to reopen it. You can rename, delete or regenerate it.

Research Papers: reformat a paper for a publisher

1. Click **Research Papers**.
2. **Your paper:** drop a PDF, Word file, LaTeX file or ZIP, or paste the text.
3. **Target format:** click one: Springer Nature journal, Springer LNCS, Elsevier, IEEE, ACM or APA 7th edition.
4. Click **Convert paper**. Wait until the badge says **Converted**.
5. Look through the tabs:
    - **Preview:** the paper in the new layout.
    - **Structure:** title, authors and sections. You can correct anything here.
    - **References:** every reference, parsed and re-formatted. Click one to fix it.
    - **Report:** what the publisher requires and how much text was preserved.
    - **Downloads:** Word, PDF, LaTeX project (for Overleaf) and BibTeX.
6. To try another publisher, click **Convert to another format**. The paper is not read again.

Brand Documents: make a branded Zensar white paper

1. Click **Brand Documents**.
2. **Brand template:** click a template card, for example *Zensar White Paper*.
    - New reference? Under **Add a reference PDF**, drop a branded PDF, give it a name and click **Learn template**. The application studies its design.
    - **Details** shows the colours, fonts and back-cover text it learned. You can edit them.
3. **Your document:** drop a PDF, Word or text file, or paste text.
4. **Length:**
    - **Match the reference** (recommended): about the same number of words and pages as the reference. Long documents are shortened and short ones lengthened by the local AI, keeping every key fact.
    - **Custom length:** choose a word count.
    - **Full document:** every word, as many pages as needed.
5. **Options:** title, cover label, number of image pages, and whether to create pictures.
6. Click **Create branded PDF**. The progress bar shows each step: reading, fitting the length, creating pictures and laying out pages.
7. Results: a PDF preview, page thumbnails, the created pictures (with the description used for each) and a check that the text arrived. Click **Download PDF**.

- **New images** creates different pictures.
- **Create again** applies a different title, label or template.

Templates

Ready-made presentation designs, built only from the brand's own colours and fonts. You can also upload your own PowerPoint template.

Media Library

All pictures, videos and audio the application can use: brand assets, your uploads and generated visuals, each with its licence.

Brand Manager

The brand's colours, fonts, logos and rules, learned from official references. You can review and edit them.

Settings

Choose the local AI model and voice, and see the privacy settings. **Offline mode** is on by default.

9. Run tests

With the project folder open in PowerShell, run the automatic checks (127 tests):

```
.venv\Scripts\python.exe -m pytest
```

All tests should pass. To check the computer itself (Python, models, Ollama, FFmpeg, GPU):

```
.venv\Scripts\python.exe scripts\check_environment.py
```

10. Stop and restart

- Stop the application by closing the **"Zensar Content Studio (close to stop)"** window or with `stop.bat`.
- To restart later, double-click `start.bat` again. It skips everything already installed and opens the application in a few seconds.
- Your projects, papers, templates and branded documents are kept in `data\`, `outputs\` and `projects\`, and survive restarts.

- To repair the installation, delete the `.venv\` folder and run `start.bat` again. Do not delete `.env` or `data\` if you want to keep your settings and work.

Troubleshooting

Symptom	Check / resolution
`start.bat` closes at once, or shows a red message	Read the message; details are in `logs\`. Run `start.bat` again after fixing the cause.
`pip` fails with "getaddrinfo failed" or a proxy error	The network blocks pypi.org. `start.bat` switches to the offline bundle by itself; to force it, `set ZCS_OFFLINE=1`. See section 6.
Address / port already in use	`start.bat` picks the next free port. To free port 8000, run `stop.bat`.
Screens load but actions fail	The backend has stopped. Check `logs\server.log`, then run `start.bat` again.
"Local model unavailable"	Start Ollama (`ollama serve`) and pick an installed model in **Settings**, or use the fast extractive mode.
Scanned PDF rejected	Install OCR with `.venv\Scripts\python.exe -m pip install rapidocr_onnxruntime`, then retry.
Video has no voice	No offline voice was found. Windows voices are under Windows **Settings → Time & Language → Speech**.
Picture creation fails or is skipped	Check that step `[6]` finished. Run `.venv\Scripts\python.exe scripts\fetch_models.py --images`.
Fonts look different from the brand	Copy the licensed Graphik files into `brands\zensar\assets\fonts\`.
Slow generation	Use the smaller model `qwen2.5:3b`, or run on the GPU server.

11. Project File Reference

This section describes every file and folder in the project, in plain words.

Tech stack

1. Frontend (the screens)

Technology	Purpose
React 18 (`react` , `react-dom`)	Builds the screens.
TypeScript	The language the screens are written in.
Vite 6	Builds the screens and runs the development server.
React Router (`react-router-dom`)	Moves between pages.
Tailwind CSS 4	Look and layout.
Lucide React (`lucide-react`)	Icons.
Node.js with `npm`	Only needed to rebuild the screens; a ready-built copy is in `frontend\dist` .

2. Backend and API

Technology	Purpose
Python 3.10–3.12 (3.12 installed by `start.bat`)	The language of the engine.
FastAPI (`fastapi`)	Web framework and API routes.
Uvicorn (`uvicorn`)	The web server.
Pydantic (`pydantic`)	Checks all data, including the AI's answers.
SQLite	Small local database for projects, jobs and settings.
`python-dotenv`	Reads the `.env` settings.

3. Reading documents

Technology	Purpose
PyMuPDF (`pymupdf`)	Reads PDFs: text with position and font, pictures, drawings.
`python-docx` , `python-pptx`	Read Word and PowerPoint files.
BeautifulSoup, `lxml`	Read web pages.

Technology	Purpose
RapidOCR (`rapidocr_onnxruntime` , optional)	Reads text from scanned pages.

4. Local AI and search

Technology	Purpose
Ollama with **Qwen 2.5** (7B or 3B)	The local writing AI: plans slides, shortens or lengthens text, describes pictures.
`qwen2.5vl:3b` (optional)	Local vision AI that describes demo-video scenes.
all-MiniLM-L6-v2 on ONNX Runtime	Turns text into numbers that capture its meaning.
FAISS (`faiss-cpu`)	Finds the passages most similar in meaning to an instruction.
Stable Diffusion **DreamShaper 8 LCM** (`diffusers` , `torch`)	Creates pictures for branded documents, on the GPU when available.

5. Output and media

Technology	Purpose
`python-pptx`	Writes editable PowerPoint files.
ReportLab (`reportlab`)	Writes PDFs: papers and branded documents.
Pillow, Matplotlib	Pictures, previews and charts.
FFmpeg (`imageio-ffmpeg`)	Builds the MP4 videos.
Windows SAPI voices (optional Piper)	Offline narration.
`fontTools`	Reads and measures fonts.

6. Research papers

Technology	Purpose
`citeproc-py` with official CSL styles	Formats citations and reference lists in each publisher's style.
`bibtexparser`	Reads and writes BibTeX.
`pylatexenc`	Converts between LaTeX and plain text.

Technology	Purpose
Jinja2 (`jinja2`)	Fills in the publisher's LaTeX templates.

Root

File / Directory	Description
`start.bat`	**The one file you double-click.** Installs whatever is missing, then starts the application and opens the browser.
`stop.bat`	Stops the running application.
`README.md`	Technical introduction to the project (for developers).
`.env` / `.env.example`	Local settings (AI model, port, folders). `start.bat` copies the example to `.env` the first time. Never commit `.env` .
`requirements.txt`	The Python packages the application needs.
`requirements-images.txt`	Extra packages for creating pictures (image AI).
`requirements-prepare.txt`	Tools needed only once to prepare the text-understanding model (developers).
`pytest.ini`	Settings for the automatic tests.
`.gitignore`	Keeps your documents, models, logs and settings out of GitHub.
`.gitattributes`	Tells Git how to treat line endings.

docs/

File	Description
`User_Manual.pdf`	This manual.
`Project_Documentation.pdf`	What the project is, how it is built and the technology used.
`Pipeline_Explanation.pdf`	How each feature works, step by step.
`Module_Architecture.pptx`	Slides explaining the parts (modules) of the application.
`Pipeline.pptx`	Slides explaining the processing pipelines.
`demo/Zensar_Content_Studio_Demo.mp4`	The demo video.
`source/`	The text of the three PDFs, for editing. Run `scripts\make_docs_pdfs.py` to rebuild the PDFs.

scripts/

File	Description
`bootstrap.ps1`	**The real installer behind `start.bat`.** Checks and installs Python, packages, models, the screens and the AI, then starts the application.
`setup.bat`	An older setup shortcut; runs the same installation as `start.bat`.
`check_environment.py`	Checks the computer and lists what is ready and what is missing (step 8 of the setup).
`fetch_models.py`	Downloads the AI models: text understanding, and with `--images` the picture model.
`prepare_models.py`	Developer tool: converts the text-understanding model into a faster format.
`offline_bundle.py`	Packs everything into pieces for computers that can only reach GitHub, and puts them back together there.
`make_offline_bundle.bat`	Builds the offline bundle on a computer with internet.
`record_demo.py`	Records the demo video by clicking through the application automatically.
`edit_demo.py`	Cuts and speeds up the recording into the final demo video.
`make_docs_pdfs.py`	Builds the documentation PDFs from `docs\source\`.
`make_docs_decks.py`	Builds the two documentation slide decks.
`zensar_brand_research.py`	One-time tool that collected the Zensar brand facts (colours, logos) from official Zensar sources.
`smoke_pipeline.py`	Developer check: runs reading, understanding and planning on one file.
`smoke_render.py`	Developer check: makes slides and pictures from a test document.
`e2e_media_test.py`	Developer check of the media features against a running application.

backend/

File	Description
`main.py`	**Starts the server.** Defines the API (`/api/...`), connects all the parts and serves the screens to the browser.
`schemas.py`	The shared "forms" every step fills in, for example what a slide or a content plan looks like.

`backend/api/` — the doors the screens knock on

File	Description
`documents.py`	Uploading documents, demo videos, code archives and GitHub links.
`projects.py`	Projects: list, open, edit the plan, regenerate a slide, download files.
`generation.py`	Starting generation jobs and reporting progress.
`brands.py`	The Brand Manager: brand profiles, assets and reference analysis.
`templates.py`	Analysing presentation templates.
`media.py`	The Media Library.
`video.py`	The video editor: timeline, script import and rendering.
`papers.py`	Research Papers: convert, review, correct and download.
`branddocs.py`	Brand Documents: learn templates, create branded PDFs, download them.
`settings.py`	Settings and the system status shown on the Dashboard.

`backend/ingestion/` — reading files

File	Description
`base.py`	The common rules every reader follows (a plug-in system for new file types).
`pdf.py`	PDF files (OCR only for scanned pages).
`docx.py`	Word documents.
`pptx.py`	PowerPoint files (each slide becomes a page).
`html.py`	Web pages (scripts are ignored, nothing is run).
`markdown.py`	Markdown text files.
`txt.py`	Plain text files (guesses the headings).
`video.py`	Demo videos: finds the scenes, describes each one and writes a narration.
`repo.py`	Code repositories: README, technology, components, API, architecture.

`backend/extraction/` — understanding the layout

File	Description
`structure.py`	Finds the title, the sections and their types (summary, method, results…).
`ocr.py`	Reads text from scanned pages (optional, offline).

`backend/intelligence/` — finding the right facts

File	Description
`chunking.py`	Cuts documents into small passages, remembering which page each came from.
`embeddings.py`	Turns each passage into numbers that capture its meaning (local AI model).
`retrieval.py`	Finds the passages most similar in meaning to a question (vector search).
`relevance.py`	Ranks passages by how relevant they are to your instruction and drops the rest.
`verification.py`	**Checks every generated statement against its source.** Numbers must match exactly.
`text.py`	Small text helpers used by the parts above.

`backend/llm/` — talking to the local writing AI

File	Description
`__init__.py`	Gives the rest of the application the configured local AI.
`base.py`	The common interface, so any local AI model can be plugged in.
`ollama.py`	Talks to Ollama on this computer only (refuses outside servers).
`prompts.py`	Loads the AI instructions from the `prompts` folder.
`structured_output.py`	Never trusts raw AI output: extracts it, checks it, repairs it or asks again.
`prompts/*.txt`	The instructions given to the AI: system rules, slide writer, visual planner, narration writer, instruction reader.

`backend/planning/` — deciding what goes where

File	Description
`content_contract.py`	Turns your instruction into a clear specification (how many slides, audience, focus).
`content_planner.py`	Builds the slide-by-slide plan from the relevant, checked facts.
`slide_planner.py`	Rules for each slide layout (what fits, safe fall-backs).
`visual_planner.py`	Decides which slides get a chart, diagram or picture (charts only from real table numbers).
`video_planner.py`	Decides how long each scene lasts in the video.
`script.py`	Reads a narration script you supply and splits it per slide.

`backend/pipeline/` — running the jobs

File	Description
`jobs.py`	Runs jobs one at a time in the background and records progress.
`orchestrator.py`	The step-by-step recipe for each job: read, understand, plan, check, draw.
`sources.py`	Combines several sources (documents, videos, repositories) in one project.
`outputs.py`	Writes every result file: PowerPoint, PDF, pictures and video.
`video.py`	Builds and renders the project video.

`backend/rendering/` — drawing slides, pictures and videos

File	Description
`scene.py`	One drawing description per slide that PowerPoint, picture and SVG output all draw identically.
`layouts.py`	Decides where everything goes on a slide, using the brand's sizes and colours.
`components.py`	Re-usable building blocks: title bars, bullet lists, number tiles, diagrams.
`grid.py`	The invisible grid that keeps everything aligned.
`motifs.py`	Zensar's shape language (circle, square, triangle) for decorations.

File	Description
`experience.py`	Zensar "Experience" designs: cover, divider and gradient styles.
`charts.py`	Brand-styled charts.
`images/raster.py`	Draws slides as PNG/JPG pictures (previews, PDF, video frames).
`images/svg.py`	Draws slides as SVG vector images.
`ppt/generator.py`	Builds the editable PowerPoint file.
`video/timeline.py`	The editable list of video clips (trim, crop, speed, narration).
`video/composer.py`	Turns the timeline into an MP4 with motion, narration, music and subtitles.
`video/motion.py`	The animations: titles fly in, diagrams build, numbers count up.
`video/narration.py`	Turns text into speech with offline Windows voices (or Piper).
`video/audio.py`	Mixes narration and music, lowering the music under speech.
`video/storyboard.py`	Simple video assembly from slide pictures and narration.
`video/ffmpeg.py`	Finds and runs FFmpeg, the video tool.

`backend/branding/` — the brand rules

File	Description
`brand_profile.py`	Loads and saves brand profiles (colours, fonts, logos). **Never invents a value.**
`theme.py`	Turns a brand profile into exact colours, fonts and sizes, with neutral fall-backs when a value is missing.
`fonts.py`	Finds the fonts installed on the computer and measures text.
`presets.py`	Presentation designs (Corporate, Bold, …) built from brand colours.
`template_parser.py`	Reads a PowerPoint template and suggests brand values from it.
`reference_analyzer.py`	Reads brand pictures and suggests colours, which you then confirm.
`social.py`	The Zensar "Experience" design language measured from Zensar's LinkedIn posts.

`backend/media/` — pictures and media

File	Description
`library.py`	The Media Library storage. Each file is stored once, with its details and licence.
`treatment.py`	Simple picture edits: crop, brightness, brand tint, borders.
`online.py`	**Optional** licensed image search (off by default; sends only generic words).
`query.py`	Builds safe search words that never contain document text.

`backend/vision/` — understanding videos

File	Description
`keyframes.py`	Finds where the scenes of a screen recording change.
`describe.py`	Describes each scene picture with the local vision AI.

`backend/services/`, `backend/storage/`, `backend/utils/`

File	Description
`services/documents.py`	Stores an uploaded document and reads it once, keeping the results.
`services/github.py`	Downloads a GitHub repository as a ZIP. The only feature that goes online, and only when you ask.
`storage/database.py`	Saves and loads projects, jobs and settings in the local SQLite database.
`storage/models.py`	The tables of that database.
`utils/config.py`	Reads the settings from `.env`: folders, port, AI model.
`utils/files.py`	Safe file names and paths.
`utils/logging.py`	Writes the log. It records counts and timings, **never** document text.
`utils/network_guard.py`	**Privacy guard:** blocks every connection except to this computer.
`utils/progress.py`	Reports the progress of long steps to the screen.

backend/papers/ — Research Papers

File	Description
service.py	Runs a conversion from start to finish and stores the results.
model.py	The description of a paper (title, authors, sections, references) shared by every reader and writer.
formats.py	The rules of each publisher (IEEE, Springer, Elsevier, ACM, APA): page size, columns, fonts, numbering.
structure.py	Works out the title, authors, abstract and sections.
references.py	Splits the reference list and reads each entry into fields (authors, year, title, journal…).
citations.py	Finds citations such as [3] or (Smith, 2020) and links them to references.
crossrefs.py	Links "Table 2", "Fig. 3", "Section 2.1" so they renumber in the new format.
cite_render.py	Formats citations and references in the publisher's official style.
math_render.py	Draws LaTeX formulas as pictures for Word and PDF.
checks.py	The report: the publisher's rules and how much of the text was kept.
readers/pdf.py	Reads PDF papers: columns, headings, broken words, formulas, tables, figures.
readers/docx.py	Reads Word papers, including Word equations.
readers/latex.py	Reads LaTeX papers (.tex or a .zip project).
readers/omml.py	Converts Word equations into LaTeX.
readers/text.py	Reads Markdown, plain text and pasted text.
writers/tex.py	Writes the LaTeX project in the publisher's official class.
writers/word.py	Writes the Word file in the publisher's layout.
writers/pdf.py	Writes the PDF in the publisher's layout.
writers/bibtex.py	Writes the references as a BibTeX file.
writers/common.py	Rules shared by all writers: numbering, captions, statements.
styles/*.csl, styles/NOTICE.md	The publishers' official citation styles and their licence notice.

`backend/branddocs/` — Brand Documents

File	Description
`service.py`	Runs a brand-document job from start to finish, and stores templates and results.
`reference.py`	**Learns a template** from a reference PDF: page types, colours, fonts, layout, logos and shapes.
`model.py`	The description of a learned template.
`content.py`	Reads your document into sections, paragraphs, lists, tables and the conclusion.
`fit.py`	**Fits the length** to the reference: shortens or lengthens with the local AI, checks every number and trims to size.
`imagegen.py`	Writes picture descriptions from the content, creates the pictures and colour-matches them to the brand.
`imagegen_worker.py`	The separate helper program that runs the picture AI (so its memory is freed afterwards).
`compose.py`	**Lays out the PDF pages** like the reference: cover, cards, columns, image pages, conclusion and back cover.
`fonts.py`	Uses the brand's own font if installed, or the brand's fall-back font.

`frontend/`

File / Directory	Description
`index.html`	The empty page the screens are loaded into.
`package.json`, `package-lock.json`	The list of building blocks for the screens.
`vite.config.ts`	Build settings. Passes `/api` requests to `http://127.0.0.1:8000` during development.
`tsconfig.json`	TypeScript settings.
`public/favicon.svg`	The small icon in the browser tab.
`dist/`	The **ready-built screens**, so computers without Node.js work too.

frontend/src/

File	Description
main.tsx	Starts the screens.
App.tsx	Decides which page shows for which address (the map of the application).
index.css	Shared look: fonts and colours.
layouts/AppLayout.tsx	The frame around every page: left menu and page header.
services/api.ts	Sends every request from the screens to the backend.
types/index.ts	Descriptions of the data the screens receive.
hooks/useAsync.ts	Helper that loads data and shows progress.
utils/format.ts	Helpers for dates and sizes ("2 minutes ago").

frontend/src/pages/

File	Description
Welcome.tsx	First-run welcome and setup wizard.
Dashboard.tsx	Home: recent projects and system status.
NewProject.tsx	Create a project: sources, template, instruction, outputs.
Projects.tsx	The list of projects.
ProjectDetail.tsx	One project: presentation, images, video, sources, plan, files.
ResearchPapers.tsx	Convert a paper: upload, choose a format, list of conversions.
PaperDetail.tsx	One converted paper: preview, structure, references, report, downloads.
BrandDocuments.tsx	Brand Documents: templates, upload, length, options, list.
BrandDocDetail.tsx	One branded PDF: preview, images, check, create again.
Templates.tsx	Presentation templates.
MediaLibrary.tsx	The Media Library.
Brands.tsx	The list of brands.
BrandEditor.tsx	Edit one brand (colours, fonts, logos, references).
Settings.tsx	Settings and system status.

frontend/src/components/

File	Description
ui.tsx	Buttons, cards, badges, alerts, dialogs and tabs used everywhere.
FileDrop.tsx	The "drop files here" box.
JobProgress.tsx	The progress bar for running jobs.
OutputPanels.tsx	Result panels for presentations and pictures.
Presenter.tsx	Full-screen presentation mode.
SlideEditor.tsx	Editing a single slide.
TemplateGallery.tsx	The gallery of design templates.
MediaPicker.tsx	Choosing pictures and clips from the Media Library.
VideoEditor.tsx	The video timeline editor.

brands/zensar/

File / Directory	Description
brand.json	Brand name and general facts.
colors.json	The Zensar colours (indigo, coral, …) from official sources.
typography.json	Fonts: Graphik, with Arial as the fall-back.
components.json, grid.json, layouts.json, shapes.json	Rules for components, the layout grid, slide layouts and shapes.
media.json	How pictures are treated in the brand.
social.json	The "Experience" design language measured from Zensar's LinkedIn posts.
video.json, voice.json	Video and narration preferences.
assets/logo/	The Zensar logo (white and dark, PNG and SVG).
assets/fonts/	Put the licensed Graphik font files here (never uploaded to GitHub).
references/	Official brand reference images.

`tests/`

File	Description
`conftest.py`	Prepares a throw-away data folder for the tests.
`test_ingestion.py`	Reading every file type.
`test_planning.py`	Instruction understanding and slide planning.
`test_relevance_and_contract.py`	Choosing relevant facts and following the instruction.
`test_brand_and_rendering.py`	Brand rules and drawing slides.
`test_design_language.py`	The Zensar design language.
`test_media_features.py`	The Media Library and picture features.
`test_video.py`	Video building and editing.
`test_api_and_privacy.py`	The API, and that nothing goes online.
`test_papers.py`	Research Papers: reading, references, all formats.
`test_branddocs.py`	Brand Documents: templates, length fitting, tables, layout.
`fixtures/`	Generated sample files used by the tests.

Folders created while the application runs (not in GitHub)

Folder	Description
`.venv/`	The application's private Python with its packages.
`data/`	Uploads, projects, papers, brand templates, branded documents and the database (`app.db`).
`outputs/` , `projects/`	Generated presentations, pictures and videos.
`models/`	The downloaded AI models.
`logs/`	Setup and server logs (useful when something fails).
`offline/`	The offline bundle (only on networks that block pypi.org).

12. Sample Data Information

The `samples\` folder contains example inputs so the application can be tried at once. They are fictional and safe to share. The samples are organised into three groups.

Group 1 - Showcase project (`samples/`)

- **Purpose:** one fictional product, *InvoiceFlow*, that exercises every feature of the Projects tool.
- **Content:**

File	Description
`InvoiceFlow_Showcase.docx`	A product document with headings, lists, tables and figures. Also used for the Brand Documents demo.
`demo_invoiceflow.mp4`	A short product demo screen recording (for video understanding).
`architecture_diagram.png` , `brand_backdrop.png`	Sample pictures for the Media Library.
`background_music.mp3`	Sample background music for videos.
`narration_script.md`	A sample narration script for the video editor.
`make_showcase.py` , `run_showcase.py`	Rebuild the samples, and create the showcase project automatically.

Group 2 - Research papers (`samples/papers/`)

- **Purpose:** one paper in each input format, for the Research Papers tool.
- **Content:**

File	Description
`sample_ieee_two_column.pdf`	A two-column IEEE-style PDF paper with formulas, tables, figures and numbered references.
`sample_author_year.docx`	A Word paper with author-year citations, e.g. (Smith, 2020).
`sample_latex_project.zip`	A LaTeX project with a `.bib` reference file.
`sample_raw_text.txt`	A paper as plain text, as if pasted.
`make_sample_papers.py`	Rebuilds the sample papers.

Group 3 - Brand reference (Brand Documents)

- **Purpose:** the design source for branded white papers.
- **Content:** a real Zensar white paper is uploaded once on the **Brand Documents** page as a reference PDF. The application learns its page types, colours, fonts, artwork and length, and stores the template in `data\branddocs\templates\` . Reference PDFs are company material and are **not** stored in GitHub; upload them again on a new computer.

Output Data

Results are written to these folders:

Folder	Contents
`outputs\<project>\`	PowerPoint, PDF, pictures and MP4 of each project.
`data\papers\<paper>\outputs\<format>\`	The converted paper: LaTeX project, Word, PDF, BibTeX and report.
`data\branddocs\docs\<document>\document.pdf`	The branded PDF, with its page previews and pictures.
`data\app.db`	The database of projects, jobs and settings.

Everything can also be downloaded from the application.